

# Tool Classification

## Contents

- 7.1. Tool Metrics
- 7.2. Reactis Hazards and Operability
- 7.3. External Error Impacts
- 7.4. Reactis Misuse

Clause 11 of ISO 26262-8 recognizes the important role tools play in the development and validation of software systems, and recommends best practices for determining the level of confidence to place in a software tool. Two important parts of tool qualification under ISO 26262 are the *tool impact level*, which measures the effect of a tool on product quality, and *tool error detection level*, which measures the likelihood that erroneous tool behavior will be detected. These two values are then combined using a table to determine the *tool confidence level* of a tool. [Section 7.1](#) gives definitions for these metrics and explains how they are measured.

An accompanying document details a Failure Modes and Effects Analysis (Reactis FMEA Version 1.7) undertaken to determine possible malfunctions of Reactis in each of the use cases described in [ISO 26262 Use Cases](#) and assess the tool impact level and tool error detection level for each possible failure. This section summarizes that analysis. The metrics used for the analysis are presented in [Section 7.1](#). [Section 7.2](#) examines possible malfunctions within Reactis. This is also known as a *hazard and operability* (hazop) study. [Section 7.3](#) examines the effects of malfunctions outside of Reactis. [Section 7.4](#) examines possible misuses of Reactis.

## 7.1. Tool Metrics

ISO 26262 defines three metrics for measuring the risk of using a tool. The first of these is the *tool impact level*, which measures the possible worst-case effect of a tool malfunction. There are only two tool impact levels:

**TI1**

[Skip to main content](#)

**TI2**

Tool could possibly introduce errors or fail to detect errors.

The second metric is the *tool detection level*, which is an estimate of the likelihood that an error will be detected. There are three tool detection levels:

**TD1**

High confidence level that error by tool will be detected.

**TD2**

Medium confidence level that error by tool will be detected.

**TD3**

Neither TD1 nor TD2 applies.

The third metric is the *tool confidence level*, which measures the confidence with which the tool can be used. There are three tool confidence levels:

**TCL1**

Tool has no impact on product quality, so no confidence in the tool is needed.

**TCL2**

Tool has a medium impact on product quality, so a medium level of confidence in the tool is needed.

**TCL3**

Tool has a high impact on product quality, so a high level of confidence in the tool is needed.

The TCL of a particular tool is determined directly from the tool impact and detection levels using the following table.

| Tool Impact | Tool Detection | Tool Confidence |
|-------------|----------------|-----------------|
| TI1         | Any            | TCL1            |
| TI2         | TD1            | TCL1            |
| TI2         | TD2            | TCL2            |
| TI2         | TD3            | TCL3            |

[Skip to main content](#)

If a tool is TCL1, no qualification is needed. For TCL2 and TCL3, the tool needs to be qualified.

## 7.2. Reactis Hazards and Operability

The hazard and operability (Hazop) study identifies possible malfunctions of Reactis. For each malfunction, the tool impact, tool detection, and tool confidence levels are determined. The results of the hazop study are presented in [Table 7.1](#). The conclusion of the hazop study is that the impact of Reactis is TI2 in all use cases, and for most use cases, there exists at least one error with a detection level of TD3, so Reactis generally has a required confidence level of TCL3. Additional details are available in the Reactis FMEA Version 1.7 document.

Table 7.1 Hazop summary for Reactis.

| Use case                                | Error                                | TI | TD | TCL |
|-----------------------------------------|--------------------------------------|----|----|-----|
| Back-to-Back test (6.1)                 | Incorrect model/code execution       | 2  | 2  | 2   |
|                                         | False output difference              | 2  | 1  | 1   |
|                                         | Undetected output difference         | 2  | 3  | 3   |
|                                         | Incorrectly reporting covered target | 2  | 3  | 3   |
|                                         | Incorrect uncovered target           | 1  | 1  | 1   |
| Detect errors in models/code (6.2, 6.4) | Incorrect model/code execution       | 2  | 2  | 2   |
|                                         | False runtime error                  | 1  | 1  | 1   |
|                                         | Undetected runtime error             | 2  | 3  | 3   |
| Measure test coverage (6.3)             | Incorrect model/code execution       | 2  | 2  | 2   |
|                                         | Incorrect uncovered target           | 2  | 1  | 1   |
|                                         | Incorrect covered target             | 2  | 3  | 3   |
| Check requirements (6.5)                | Incorrect model/code execution       | 2  | 2  | 2   |
|                                         | Incorrect uncovered target           | 2  | 1  | 1   |
|                                         | Incorrect covered target             | 2  | 3  | 3   |
|                                         | False assertion violation            | 2  | 1  | 1   |
|                                         | Undetected assertion violation       | 2  | 3  | 3   |
| Detect dead code (6.6)                  | Incorrect unreachable target         | 2  | 2  | 2   |
| Walkthroughs and inspections (6.7)      | Incorrect model hierarchy            | 2  | 2  | 2   |
|                                         | Incorrect signal trace               | 2  | 2  | 2   |
|                                         | Subsystem navigation error           | 2  | 2  | 2   |

[Skip to main content](#)

| Use case                     | Error                      | TI | TD | TCL |
|------------------------------|----------------------------|----|----|-----|
| User-constructed tests (6.8) | Model search error         | 2  | 2  | 2   |
|                              | False runtime error        | 2  | 1  | 1   |
|                              | Undetected runtime error   | 2  | 3  | 3   |
|                              | Incorrect covered target   | 2  | 3  | 3   |
|                              | Incorrect uncovered target | 1  | 1  | 1   |

## 7.3. External Error Impacts

Table 7.2 External error impact summary for Reactis.

| Use case                  | Error                | TI | TD | TCL |
|---------------------------|----------------------|----|----|-----|
| Any case requiring MATLAB | MATLAB startup error | 1  | 1  | 1   |
|                           | Simulink load error  | 1  | 1  | 1   |
| Any case                  | System shutdown      | 1  | 1  | 1   |
|                           | Insufficient RAM     | 1  | 1  | 1   |
|                           | Corrupted File       | 2  | 1  | 1   |

The impacts of external errors on Reactis are summarized in [Table 7.2](#). The takeaway conclusion is that external errors have little effect on the required confidence level for Reactis (TCL1 in all cases). See the Reactis FMEA Version 1.7 for details.

[Skip to main content](#)

## 7.4. Reactis Misuse

Table 7.3 Potential misuse summary for Reactis.

| Use case                     | Error                         | TI | TD | TCL |
|------------------------------|-------------------------------|----|----|-----|
| Any requiring MATLAB         | MATLAB version mismatch       | 1  | 1  | 1   |
| Any case using tests         | Name mismatch                 | 1  | 1  | 1   |
|                              | Incorrect name mapping        | 2  | 1  | 1   |
|                              | Invalid input                 | 2  | 1  | 1   |
| User-constructed tests (6.8) | Invalid input                 | 2  | 1  | 1   |
| Any creating tests           | Overconstrained inputs        | 2  | 2  | 2   |
|                              | Underconstrained inputs       | 2  | 3  | 3   |
| Any executing model/code     | Misconfigured error detection | 2  | 2  | 2   |
|                              | Unsaved parameter changes     | 2  | 1  | 1   |
| Any using Reactis for C      | Missing C file                | 2  | 2  | 2   |
|                              | Orphaned instrumentation code | 2  | 1  | 1   |
|                              | Incorrect architecture        | 2  | 3  | 3   |
| Any                          | Overwritten file              | 1  | 1  | 1   |
| Requirements test (6.5)      | Incorrect objective wiring    | 2  | 2  | 2   |
|                              | Orphaned objective            | 2  | 2  | 2   |
|                              | Objective inputs changed      | 2  | 1  | 1   |
|                              | Invalid objective             | 2  | 1  | 1   |

The hazards from misuse of Reactis are summarized in [Figure 7.3](#). The items listed in the *Errors* column of [Figure 7.3](#) are explained as follows.

### MATLAB version mismatch.

This error occurs when the version of MATLAB involved by Reactis does not match the

[Skip to main content](#)

detect the version mismatch, output a warning, and require user confirmation before proceeding.

**Name mismatch.**

This error occurs when the name of an input/output in a test suite does not match the name of an input/output in the current test harness. If this occurs, Reactis will detect the mismatch and require the user to manually match all items which could not be automatically matched before proceeding.

**Incorrect name mapping.**

This misuse occurs when the user maps a test suite input/output to the wrong input/output in the test harness. When this happens, there are two most likely outcomes. These are (1) the types do not match, in which case the mapping will be rejected and testing will not proceed, or (2) there will be a large number of output differences reported (and runtime errors may occur as well).

**Overconstrained inputs.**

This misuse occurs when the user selects input constraints which do not include all the possible real-world inputs for the scenario under test. The possible effects of such a mistake include poor test coverage and failing to discover input sequences which lead to runtime errors. Errors of this nature are largely impossible for Reactis to detect, but user might detect them when investigating the cause of poor test coverage.

**Underconstrained inputs.**

This misuse occurs when the user selects input constraints which allow inputs that are not possible under real-world conditions. The possible effects of such a mistake include altered test coverage (higher or lower) and runtime errors during testing. Errors of this nature are largely impossible for Reactis to detect.

**Misconfigured error detection.**

This misuse occurs when the user configures Reactis error detection in an unintended way (e.g., the user temporarily turns off detection of integer overflows and then forgets to turn it back on). This type of error can (obviously) cause Reactis to fail to detect errors which it would normally detect. It is also impossible for Reactis to prevent, as there is no way for Reactis to know the intent of the user. The error detection settings are listed in the test execution reports as a safeguard against this type of mistake. The settings can also be viewed by the user at any time while using Reactis.

**Unsaved parameter changes.**

This refers to the case where the user makes changes to testing parameters (e.g., the

[Skip to main content](#)

changes to the parameters are detected by Reactis and the user is prompted to save them before testing can proceed.

### **Overwritten file.**

This error occurs any time Reactis attempts to save a file but the file already exists. When this happens the user is warned and can either abort the save attempt or confirm their intent to replace the file contents.

### **Missing C file.**

This error occurs when one of the source files required to build the code under test is deleted or renamed. Missing files are automatically detected by Reactis and a placeholder with a warning is displayed in the GUI. Reactis will not allow testing to proceed until the missing file is resolved.

### **Orphaned instrumentation code.**

This error occurs when a `.c` file which contains instrumentation code is deleted or renamed. In this case, the instrumentation code is saved in the `.rsm` file so that it can be recovered.

### **Incorrect architecture.**

This refers to the possibility that the user accidentally selects the wrong architecture in Reactis for C. This could potentially allow testing to succeed when it should fail, but it is more likely that a runtime error will occur.

### **Incorrect objective wiring.**

This error occurs when the inputs to a Validator objective are not wired correctly (e.g., the user accidentally selects the wrong signal for one of the inputs). This type of error may be detected automatically if the signal type does not match the input type. If undetected, can lead to false positive or false negative results.

### **Orphaned objective.**

This happens when the subsystem in which a Validator objective is located is removed or renamed. The missing subsystem is detected by Reactis and the objective is disabled until the user re-wires the inputs to the objective. Since the objective is not evaluated during testing, test cases which would lead to the objective being covered will not be discovered until the user selects a new subsystem for it.

### **Objective inputs changed.**

This happens when a model component which is wired to an input of a Validator objective is modified (e.g., a signal which is wired to a Validator objective is deleted).

[Skip to main content](#)



**Invalid Objective.**

This error occurs when a Validator objective contains a syntactic or semantic error. Validator objectives are checked by Reactis when they are created and also every time Simulator starts, so such errors will most likely be caught and testing will be unable to proceed until they are fixed.